ROSSMANN

STORE SALES FORECASTING AND DEMAND PREDICTION

Presented by:

Asmaa Elsayed Mohamed Morsy
Aya Ahmed Kamal Abdulhameed
Marvel Baheeg Moneer Azmy
Yasmeen Esmail Mohamed Abdullah

BUSINESS PROBLEM STATEMENT

Rossmann relies on individual store managers for daily sales predictions, which leads to inconsistent forecasting and operational risks.
Managers also struggle to accurately measure the combined impact of complex factors such as promotions, holidays, seasonality, and nearby competition, making sales planning more difficult and less reliable.

Why Is This a Problem?

- Poor forecasting directly impacts operational efficiency
- Underestimating sales may cause product shortages and long customer waiting times
- Overestimating demand increases operational and inventory costs
- Promotions and holidays create sudden sales spikes that are difficult to predict manually
- Different store types and competition levels make forecasting even more complex across 1,115 stores

Project Objectives

- Develop an accurate machine learning model to forecast daily sales for Rossmann stores.
- Perform data cleaning, preprocessing, and feature engineering to improve model qualityand Analyze the impact of promotions, holidays, seasonality, and competition on sales performance
- Compare multiple machine learning models to identify the best forecasting approach.
- Support data-driven decision-making for inventory planning, staffing, and business operations.

DATASET DESCRIBTION

The Rossmann dataset contains historical sales records collected from 1,115 Rossmann stores, with over 1 million daily sales records spanning several years. The data combines daily operational sales information with detailed store-related attributes to help analyze the factors affecting sales performance.

Dataset Files

train.csv

- Sales (Target)
- StoreID
- Customers
- Open
- Promo
- StateHoliday (0,a,b,c)
- SchoolHoliday
- Date

store.csv

- StoreType (a,b,c,d)
- StoreID
- Assortment (a,b,c)
- CompetitionDistance
- Promo2
- Promo2SinceWeek
- Promo2SinceYear
- PromoInterval

Data Cleaning&Preprocessing

Data Preprocessing Challenges

Problem 1 : Separate Datasets

Sales data and store metadata existed in separate files

Solution:

Merged:

- **train.csv**
- **store.csv**

using:

Store ID to create one unified dataset for analysis and modeling.

Problem 2 : Missing Values

Some store-related features contained missing values because:

- Certain stores had no nearby competitors
- Some stores did not participate in Promo2 campaigns

Solution:

- **Filled competition and promotion time features with 0**
- **Replaced missing CompetitionDistance values using the Mean**
- **Replaced missing PromoInterval values with: “No Promo”**

Data Preprocessing Challenges

Problem 3 : Inconsistent Data Types

The StateHoliday feature contained:

- Integer 0
- String "0"

Although both represented normal business days, the model would treat them as different categories.

Solution:

Standardized all values into one consistent categorical format.

Problem 4 : Incorrect Date Format

The Date column was stored as a string instead of a datetime format.

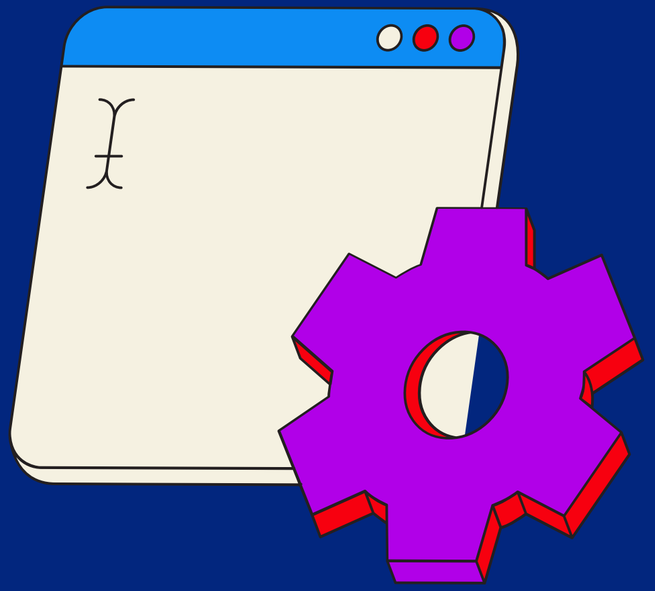
Solution:

Converted the column into:Datetime Format

This enabled:

- **Time-series analysis**
- **Seasonal feature extraction**
- **Trend analysis**

Feature Engineering & Encoding



Feature Engineering: Time& Duration

Temporal Features

- Calendar Elements: Extracted Year, Month, Day, and Week of Year from raw Date.
- Seasonal Mapping: Mapped fiscal months to capture annual shifts in consumer buying habits.

Operational Duration Metrics

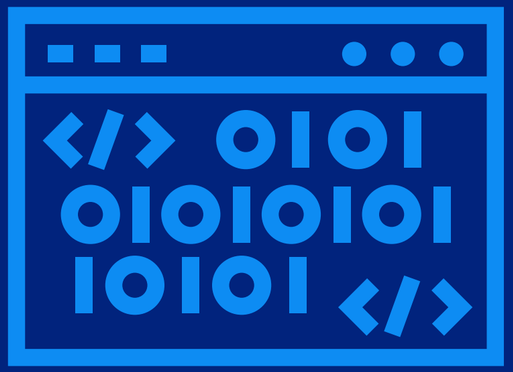
- Competition Active Duration (CompetitionOpen_month): Tracks months since the nearest competitor opened. Helps model learn if competitor impact weakens over time.
- Promotion Active Duration (Promo2Open): Measures months a store has been in Promo2. Captures customer fatigue or compounding campaign success.

Feature Engineering: Categorical Encoding

Ordinal Encoding (Hierarchical Data)

Assortment Mapping: Converted levels into ordered weights based on product variety depth:

- 'a' (Basic) ---> 0
- 'c' (Extended) ----->1
- 'b' (Extra) ------> 2



Nominal One-Hot Encoding (Non-Ordered Data)

- Targeted Columns: StoreType (layouts) and StateHoliday (holiday types) expanded into binary (0 or 1) columns.

- Leakage Prevention: Strictly .fit() on training partition only; configured handle_unknown='ignore' for live evaluation

Feature Selection & Leakage Prevention

Optimizing the Final Matrix

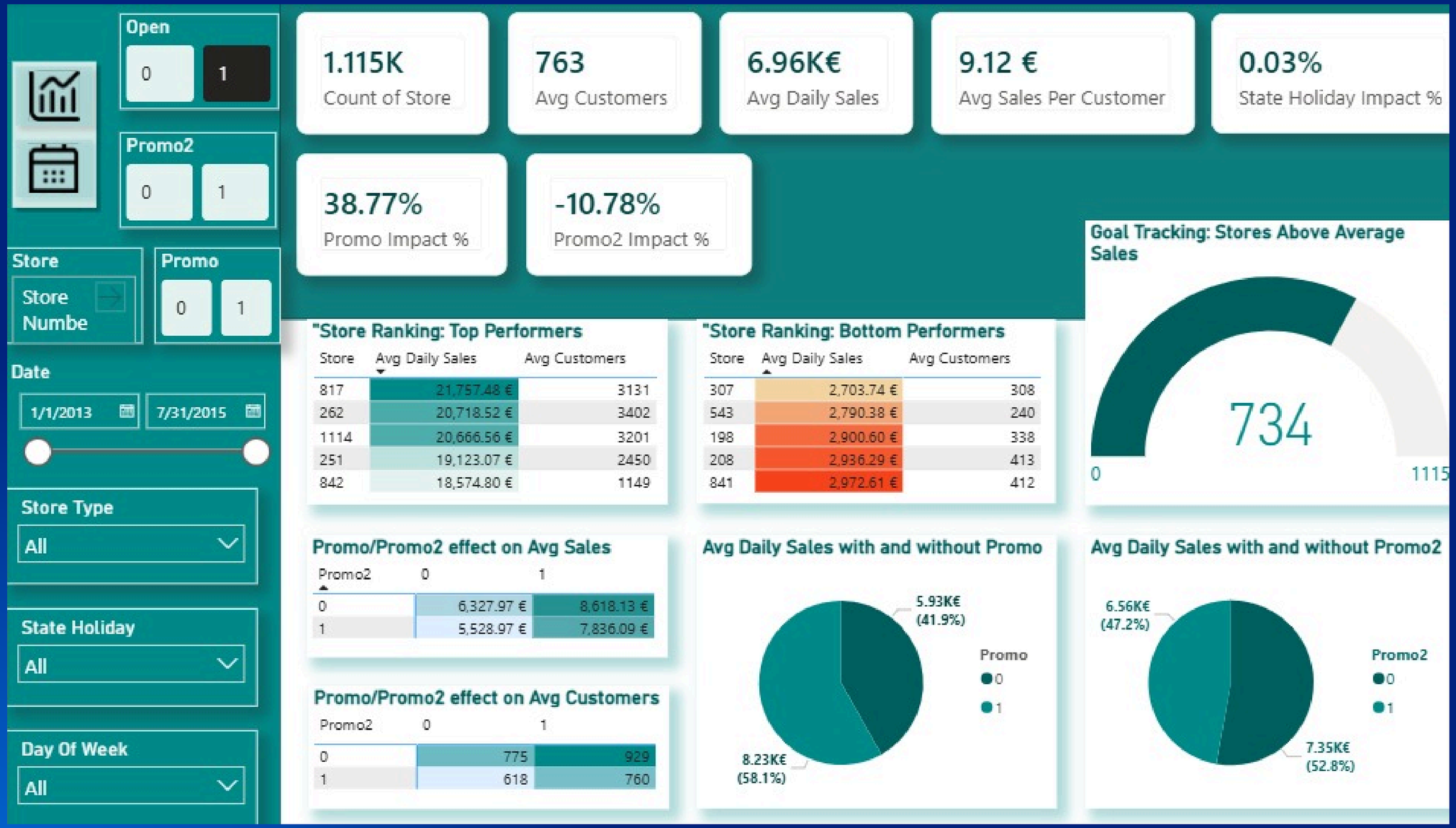
- Dropped PromoInterval: Removed because its seasonal data was already captured by duration metrics.
- Efficiency: Prevented memory overhead and ensured faster convergence for XGBoost.

CRITICAL: Removing Leaky Columns

- Dropped Customers Feature: Highly correlated with Sales, but strictly unknown during inference and missing from test.csv.
- The Goal: Mandatory step to prevent Data Leakage and guarantee realistic model evaluation in production.

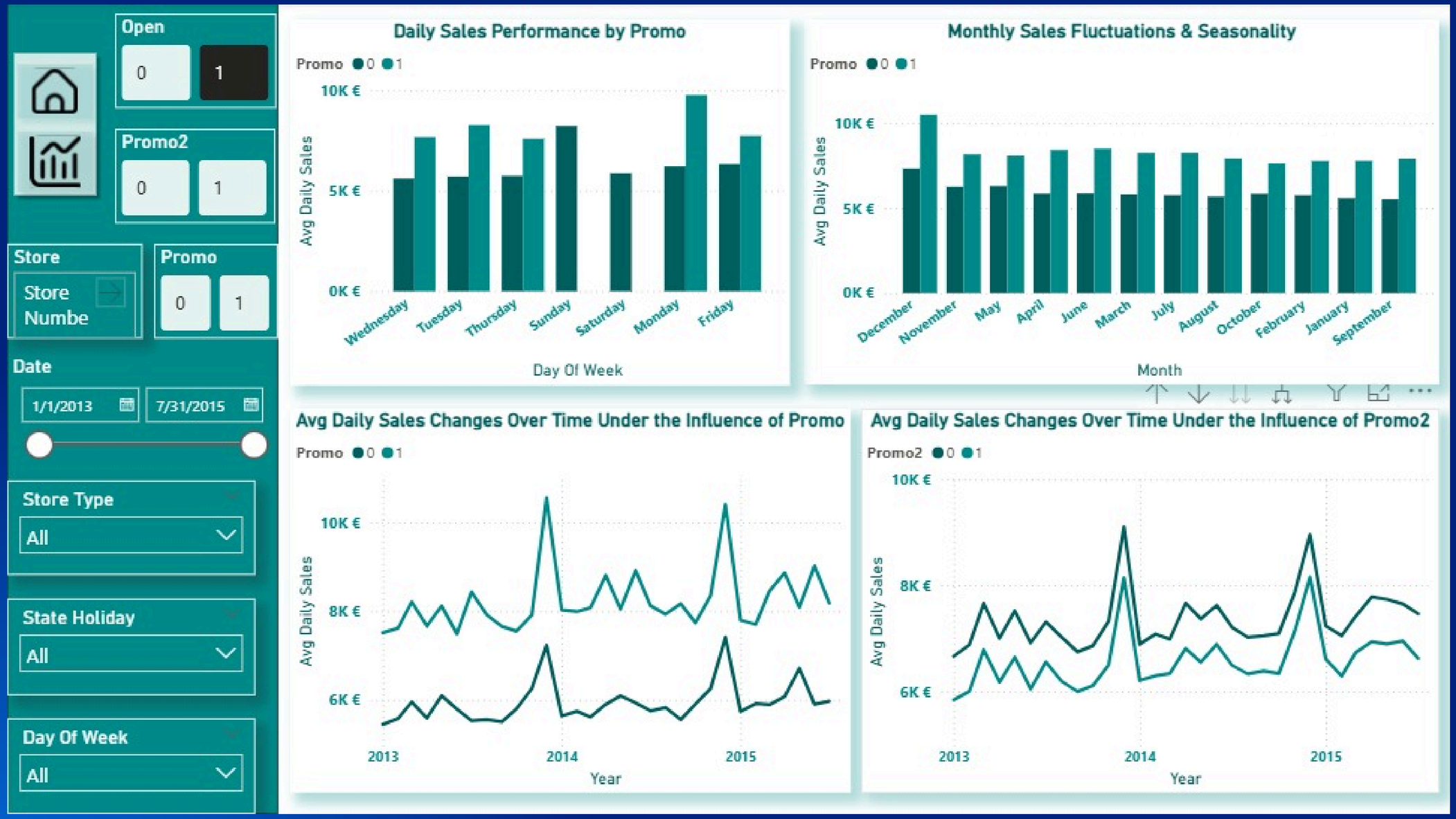
Exploratory Data Analysis (EDA)

EDA via Dashboard — High-Level KPI Metrics

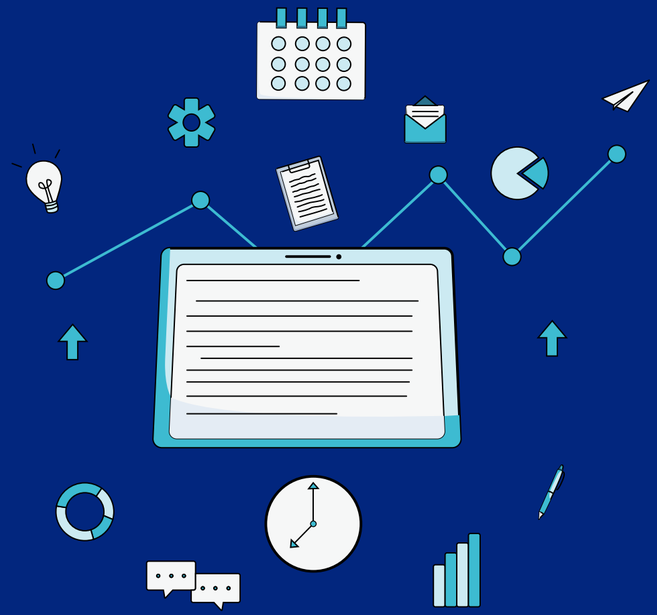


EDA via Dashboard — Promo & Holiday Efficiency

EDA via Dashboard — Time Series & Seasonality Trends



Model Training & Evaluation



Data Splitting Strategy

Problem :Risk of Data Leakage & Unrealistic Evaluation

- Traditional Random Splitting breaks the temporal order of retail sales data.
- Allowing the model to train on random past and future days simultaneously creates artificial lookahead bias.
- This results in deceptively high training accuracy but severe failure in real-world deployment.

---->

Solution: Chronological Time-Based Split

- Time-Dependent Design: Strictly maintained the sequential timeline since sales are inherently chronological.
- The Threshold: Established June 15, 2015 as the absolute cutoff date:
- Training Partition: All historical data prior to this date.
- Testing Partition: All subsequent future data reserved exclusively for evaluation.
- The Operational Goal: Mimics an authentic production environment, forcing the machine learning model to forecast true, unseen future sales.

Feature Scaling Strategy

Problem :Skewed Feature Scales & Slow Training

- Raw features and the target variable (Sales) had vastly different numeric ranges.
- This imbalance slows down gradient descent, causing unstable or extremely slow model convergence for neural networks and boosting algorithms.

---->

Solution: Min-Max Scaling & Inversion

- Normalizations: Applied MinMaxScaler to compress all variables and Sales into a strict [0, 1]range for faster, optimal model training.
- Inverse Transformation: Converted the final predictions back to the original Euro (€) scale to ensure realistic error calculations (MAE & RMSE).

Model Selection — The Deep Learning Challenge

The Problem: Is Deep Learning (LSTM) better for Time-Series?

- Sequential data often suggests using LSTMs, but Rossmann's data is highly tabular (Store Metadata, Competition distance, etc.).
- The Result: Traditional LSTM failed significantly with only 20.00% Accuracy (R^2).
- The Bottleneck: LSTMs require complex 3D sequential formatting and struggle to prioritize static tabular features compared to pure time-series data.

The Solution: Transition to Tree-Based Ensembles

- **Shifted focus from sequential deep learning to Tree-based algorithms which are inherently superior for structured, feature-rich tabular datasets.**

The Winning Solution — Tree-Based Ensembles

How to handle sudden sales spikes (Promos/Holidays) and complex non-linear relationships across 1,115 stores?
The Solution: Random Forest vs. XGBoost

Random Forest	XG Boost
• Test Accuracy (R^2): 86.56% • MAE: 773€ • RMSE: : 1111.98€	• Test Accuracy (R^2): 90.46% • MAE: 661€ • RMSE: 936€
Strengths : • **Captured nonlinear relationships well** • **Strong baseline performance**	**Strengths :** • **Better handling of complex feature interactions** • **Strong generalization capability** • **Reduced prediction error significantly**

Model Predictions Comparison

```
STATUS: Training Random Forest Model... Please wait.
SUCCESS: Model Training Completed.
================================================
Train Data Performance:
      RMSE: 349.61757010801284
      MAE: 217.469492689216
      Accuracy (R2): 0.9873459102098182
================================================
Test Data Performance:
      RMSE: 1111.98021503096
      MAE: 773.7340851321635
      Accuracy (R2): 0.8656825618187831
```

```
STATUS: Training the Professional XGBoost Model...
SUCCESS: Model Training Completed.
================================================
Train Data Performance (Winning XGBoost):
      RMSE: 654.8346165254247
      MAE: 441.2579345703125
      Accuracy (R2): 0.9556077122688293
================================================
Test Data Performance (Winning XGBoost):
      RMSE: 936.8928367214684
      MAE: 661.0055541992188
      Accuracy (R2): 0.9046505689620972
```

Hyperparameter Tuning Strategy

To extract the maximum predictive power from our algorithms, we bypassed default parameters and implemented two distinct optimization paths:

RandomizedSearchCV (with Random Forest)

Why we used it:

- Random Forest models take a significant amount of time to train on large data. RandomizedSearch was chosen for speed; it evaluates a fixed number of random parameter combinations from a broad distribution, finding an excellent approximate setup without crashing hardware resources.
- Best Key Parameters Tuned: 'n_estimators': 100, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt', 'max_depth': 30

Hyperparameter Tuning Strategy

GridSearchCV (with XGBoost)

Why we used it:

- Since XGBoost was our clear frontrunner, we chose the most meticulous method available. GridSearch systematically tests every single permutation of the parameter grid to guarantee we locked in the absolute mathematical global optimum.

- Best Key Parameters Tuned: 'learning_rate': 0.02, 'max_depth': 10, 'n_estimators': 2000, 'objective': 'reg:squarederror', 'subsample': 0.9

MLOps & Experiment Tracking

MLOps & Experiment Tracking

MLflow Integration

- MLflow was used to track and manage the machine learning experiments throughout the project lifecycle.

Modular Project Architecture

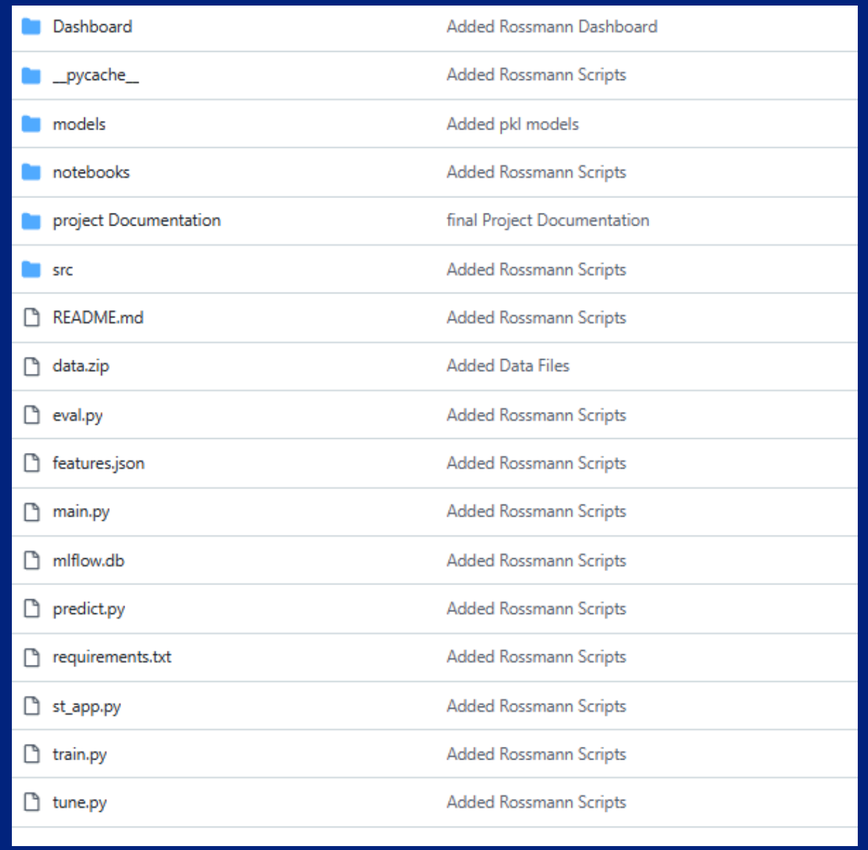
The project was organized into separate modules for:

- Data preprocessing
- Feature engineering
- Model training
- Hyperparameter tuning
- Prediction pipeline
- Streamlit dashboard

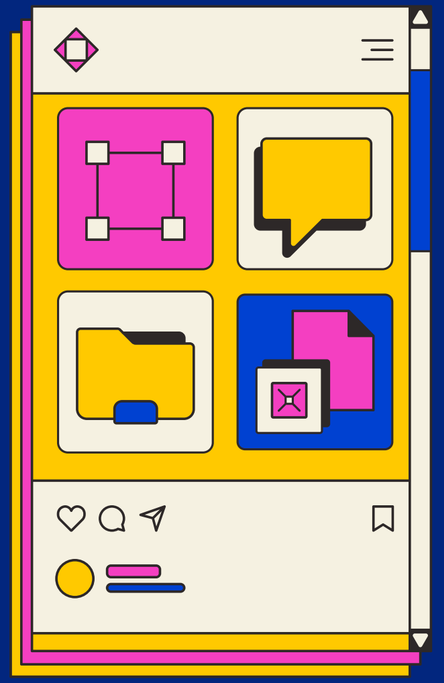
Benefit

Improved:

- Code organization
- Scalability
- Maintainability



Model Deploymemt Via Streamlit



Model Deploymemt Via Streamlit

- The final forecasting models were deployed using:Streamlit Dashboard to provide real-time sales prediction through an interactive user interface.

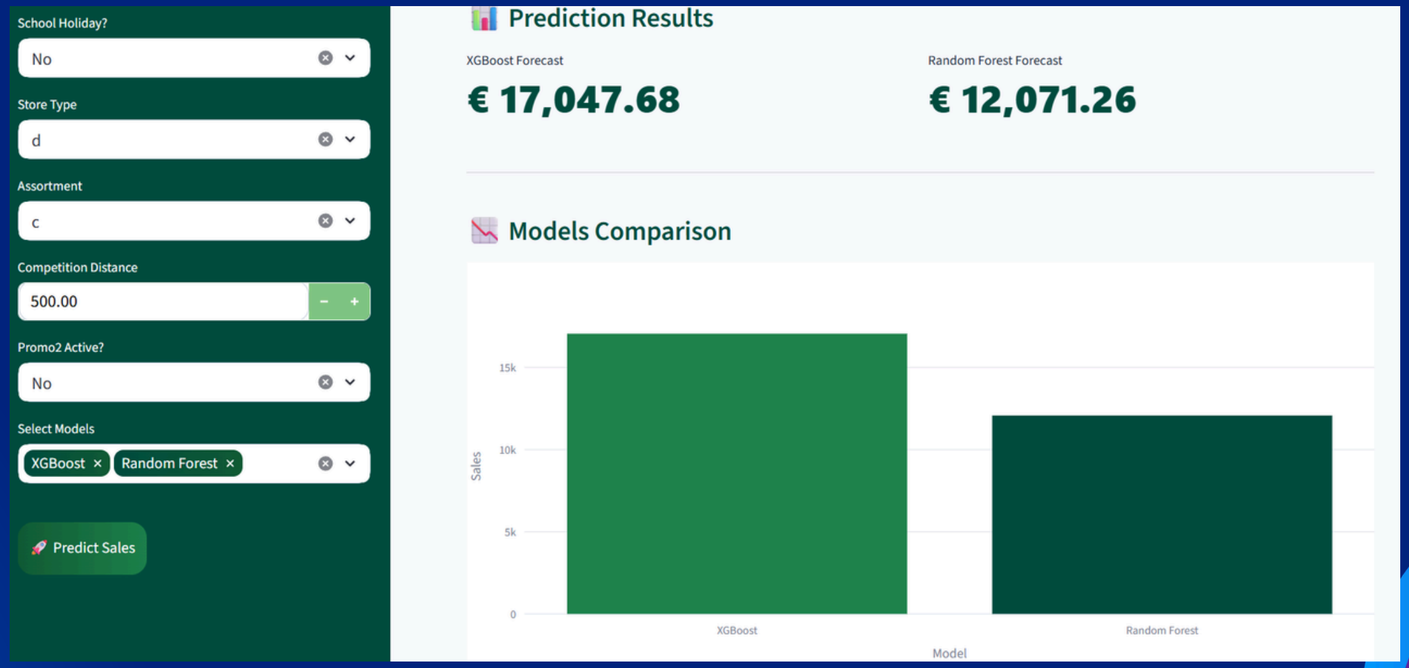
Dashboard Features

- Sales prediction for future dates
- Store and promotion configuration
- Model comparison (XGBoost & Random Forest)
- Interactive visualizations

Prediction Pipeline

User Inputs
→ Feature Processing
→ Trained Model
→ Sales Forecast
→ Dashboard Output

Model Predictions Comparison



THANK YOU !!!